

Infinite Precision Arithmetic in Java

Tavva Dinesh Reddy - CS24BTECH11061
Indian Institute of Technology Hyderabad

May 2, 2025

Contents

1	Introduction	1
2	Design	1
2.1	Design Decisions	1
2.2	File Structure	2
2.3	Class Diagram (UML)	3
3	Snapshot of Git Commits	4
4	Core Logic of Implementation	5
4.1	Overview of AInteger.java	5
4.2	Overview of AFloat.java	5
5	Limitations	5
6	Verification Approach	6
7	Key Learnings	6
8	Conclusion	6

1 Introduction

This project implements infinite precision arithmetic operations in Java, supporting calculations on arbitrarily large integers and floating-point numbers. The project overcomes the limitations of Java's native numeric types by implementing all operations using string manipulation.

2 Design

2.1 Design Decisions

We chose a string-based internal representation for all numbers to allow unbounded size. Two core classes were implemented:

- `AInteger`: Handles arbitrary-length integers.
- `AFloat`: Extends `AInteger` to support floating-point arithmetic.

2.2 File Structure

```
project-root/
|
|-- arbitraryarithmetic/
|   |--> AFloat.java
|   |--> AInteger.java
|
|-- reports/
|   |--> report.pdf
|-- build/
|   |--arbitraryarithmetic/
|       |--> AFloat.class
|       |--> AInteger.class
|   |--> MyInfArith.class
|   |--> aarithmetic.jar
|
|-- Dockerfile
|-- .Dockerignore
|-- MyInfArith.java
|-- README.md
```

2.3 Class Diagram (UML)

AInteger
-value : String
+AInteger() +AInteger(value : String) +copy(other : AInteger) : AInteger +toString() : String +add(other : AInteger) : AInteger +subtract(other : AInteger) : AInteger +multiply(other : AInteger) : AInteger +divide(other : AInteger) : AInteger +stringAdd(s1 : String, s2 : String) : String +stringSubtract(s1 : String, s2 : String) : String +stringMultiply(s1 : String, s2 : String) : String +stringDivide(dividend : String, divisor : String, quotient : String) : String +compareString(s1 : String, s2 : String) : int +validString(s : String) : boolean +parse(s : String) : AInteger +removeLeadingZeros(s : String) : String

AFloat
-value : String
+AFloat() +AFloat(value : String) +copy(other : AFloat) : AFloat +toString() : String +add(other : AFloat) : AFloat +subtract(other : AFloat) : AFloat +multiply(other : AFloat) : AFloat +divide(other : AFloat) : AFloat +addStringFloat(s1 : String, s2 : String) : String +subtractStringFloat(s1 : String, s2 : String) : String +multiplyStringFloat(s1 : String, s2 : String) : String +divideStringFloat(dividend : String, divisor : String, precision : int) : String +normalizeFloatString(s : String) : String +getIntegerPart(s : String) : String +getFractionalPart(s : String) : String +hasDecimal(s : String) : boolean +countFractionalDigits(s : String) : int +adjustDecimalPoints(s1 : String, s2 : String) : String[] +shiftDecimalPointRight(s : String, shiftRightBy : int) : String +shiftDecimalPointLeft(s : String, shiftLeftBy : int) : String +removeLeadingZeros(s : String) : String +validString(s : String) : boolean +parse(s : String) : AFloat

3 Snapshot of Git Commits

Commit Hash	Author	Date	Message
5558f9b	dineshtavva23	2025-05-01	added build.xml file
a0674ab	dineshtavva23	2025-04-30	wrote python file for compiling running the code
2ff6668	dineshtavva23	2025-04-30	added exception for invalid string inputs for both float and int
f47f0f1	dineshtavva23	2025-04-30	changed file structure and added MyInfArith.java file
9cacb6e	dineshtavva23	2025-04-30	changed file structure and also added aarithmetic.jar
b633d09	dineshtavva23	2025-04-30	handled division by zero error and now you can enter positive integers without + sign also
fdd29f9	dineshtavva23	2025-04-30	handled division by zero error and now you can enter positive integers without + sign also
c932c75	dineshtavva23	2025-04-29	handled division by zero error and now you can enter positive integers without + sign also
6dd78fd	dineshtavva23	2025-04-18	changed variable names and added exception cases where string does not contain '.' and replaced strings with string builder objects
15ad2e8	dineshtavva23	2025-04-15	added copy constructors
cb935e9	dineshtavva23	2025-04-13	added divide functions which perform division of floats
7079c75	dineshtavva23	2025-04-12	added addition, difference, multiplication functions to AFloat
e9ba355	dineshtavva23	2025-04-12	added division function, added corner cases to compare_string, resolved bug in subtraction function
5424b83	dineshtavva23	2025-04-11	added multiply function and corrected padding issue in add function
9b7ea72	dineshtavva23	2025-04-11	added subtraction function for Integers class
5ae1306	dineshtavva23	2025-04-11	overloaded sum function in AInteger class
d6062f4	dineshtavva23	2025-04-08	created files

4 Core Logic of Implementation

4.1 Overview of AInteger.java

The `AIInteger` class handles arbitrary-length integers and supports four fundamental arithmetic operations: addition, subtraction, multiplication, and division. Internally, numbers are stored as strings, and all operations simulate manual digit-by-digit arithmetic.

- **Addition/Subtraction:** These methods compare operand signs and delegate to helper functions `stringAdd` and `stringSubtract`, which perform digit-wise operations with carry or borrow handling.
- **Multiplication:** Simulates grade-school multiplication, creates partial products, and sums them using `stringAdd`.
- **Division:** Uses long-division by comparing digit segments of the dividend to the divisor and building the quotient.
- Sign handling ensures correctness for positive/negative values. Division by zero is handled gracefully.

4.2 Overview of AFloat.java

The `AFloat` class extends `AIInteger` to support floating-point arithmetic using strings. It aligns decimal points and reuses integer logic after stripping decimals.

- **Addition/Subtraction:** Aligns decimals, pads with zeros, performs integer-like operations, and repositions the decimal point.
- **Multiplication:** Removes decimals, multiplies as integers, then restores the decimal.
- **Division:** Removes decimals, uses integer division, and repositions the decimal in the result. Division-by-zero is safely handled.
- Results are normalized by removing unnecessary zeros and formatted as valid `AFloat` objects.

5 Limitations

- **High memory usage** for large numbers due to string-heavy implementation.
- **No scientific notation support** like 1.23×10^5 .
- **Suboptimal performance** for large inputs due to $O(n^2)$ algorithms.
- **String operations are inefficient** compared to lower-level memory handling.
- **No support for exponentiation or modulus**, which can be added in the future.

6 Verification Approach

- **Edge cases tested:** Included negative inputs, handling of leading/trailing zeros, and division by zero scenarios to ensure robustness.
- **Validation against standard libraries:** Verified correctness of operations (especially for large integer and floating-point division) by comparing results with outputs from standard libraries like `BigInteger` and `BigDecimal`.
- **Example test case:**
 - **Input:** `java MyInfArith float div 8792726365283060579833950521677211.0 493835253617089647454998358`
 - **Expected Output:**
17804979.091469989302961159520087878533680521806036711679246
839841308153986092362737855068303554024486248084164734360677
209716426077098087105970442196627540469031041948493652479229
112208111436434654956

7 Key Learnings

- Understood the significance of using Git for version control, collaboration, and maintaining code history.
- Learned how to effectively organize code in large-scale projects for better readability and maintainability.
- Explored new debugging techniques to identify and fix issues more efficiently.
- Realized the importance of meaningful variable names in improving code clarity, especially in larger codebases.
- Gained hands-on experience with creating and pushing Docker images, and understood their role in containerization and deployment.

8 Conclusion

This project shows how we can do very large number calculations in Java by using strings instead of normal numbers. It can correctly perform basic math operations like addition, subtraction, multiplication, and division, even for decimal numbers. It also handles special cases, such as wrong inputs or dividing by zero, in a safe way.